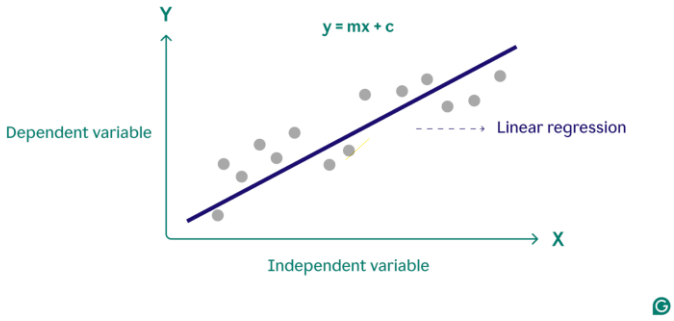
Regression

A Regression model is a fundamental supervised machine learning algorithm used for regression tasks, which means it predicts a continuous numeric value (like price, temperature, or age).

The core idea is to model the relationship between a dependent variable (the target, usually denoted as y) and one or more independent variables (the features, denoted as x_i) by fitting a line or a hyperplane to the observed data.

Simple Linear Regression (One Feature)



If there is only one independent variable x, the model is called Simple Linear Regression, and the equation is:

$$\hat{y} = mx + c$$

- $\hat{y}$: The predicted value (the output).
- c: The bias (or y-intercept), which is the predicted value of $\hat{y}$ when x is zero.
- m: The slope (or weight) of the feature, which determines how much $\hat{y}$ changes for a unit change in x.
- x: The feature (the input variable).

Once this line is chosen, we use it to predict values of y for given inputs x. But to know whether our chosen line is good or bad, we need a way to measure how far our predictions $\hat{y}_i$ are from the actual values y_i. For this purpose, we use a cost function. In linear regression, the most common cost function is the Mean Squared Error (MSE), defined as

$$J = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2.$$

This cost function gives a single number that represents the total error of the model. A smaller value of J means the line fits the data better.

To reduce the cost and find the best values of m and c, we use gradient descent. Gradient descent needs the gradients (partial derivatives) of the cost function with respect to the parameters. These gradients tell us how the cost changes when we slightly change m or c. The gradient with respect to m is

$$\frac{\partial J}{\partial m} = \frac{1}{n}\sum_{i=1}^{n}\big((mx_i + c) - y_i\big)x_i,$$

and the gradient with respect to c is

$$\frac{\partial J}{\partial c} = \frac{1}{n}\sum_{i=1}^{n}\big((mx_i + c) - y_i\big).$$

The gradients show the direction in which we should adjust m and c so that the cost decreases. By repeatedly updating

$$m := m - \alpha\frac{\partial J}{\partial m}, \qquad c := c - \alpha\frac{\partial J}{\partial c},$$

where α is the learning rate, the algorithm gradually finds the values of m and c that minimize the error. This is how linear regression learns the best-fitting line through the given data.

Vectorized Form of Linear Regression (Batch Gradient Descent)

In the vectorized approach, instead of computing predictions and gradients for each data point individually, we use matrix operations to compute everything at once. This makes the computation faster and more efficient, especially for large datasets.

1. Design Matrix

We first construct the input matrix X:

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

The first column of 1's corresponds to the intercept term c.

This makes calculation easier because:

$$w = \begin{bmatrix} c \\ m \end{bmatrix}$$

2. Vectorized Prediction

The prediction vector is computed as:

$$\hat{y} = Xw$$

This automatically calculates all predicted values at once.

3. Vectorized Cost Function

The cost function in vector form becomes:

$$J(w) = \frac{1}{2n}(Xw - y)^T(Xw - y)$$

This replaces the summation with a matrix multiplication.

4. Vectorized Gradients

Batch gradient descent requires computing the gradients of the cost with respect to the parameters w.

In vectorized form:

$$\nabla_w J = \frac{1}{2n} \cdot 2X^T(Xw - y) = \frac{1}{n}X^T(Xw - y)$$

5. Vectorized Gradient Descent Update Rule

$$w := w - \alpha \nabla_w J = w - \alpha \frac{1}{n}X^T(Xw - y)$$

Multiple Linear Regression

The prediction (hypothesis) is written as:

$$\hat{y}_i = w_0 + w_1 f_{i1} + w_2 f_{i2} + \cdots + w_d f_{id}$$

1. Design Matrix with Features

For n samples and d features, the matrix X becomes:

$$X = \begin{bmatrix} 1 & f_{11} & f_{12} & \cdots & f_{1d} \\ 1 & f_{21} & f_{22} & \cdots & f_{2d} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & f_{n1} & f_{n2} & \cdots & f_{nd} \end{bmatrix}$$

The parameter vector:

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

2. Vectorized Prediction:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

3. Vectorized Cost Function:

$$J(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

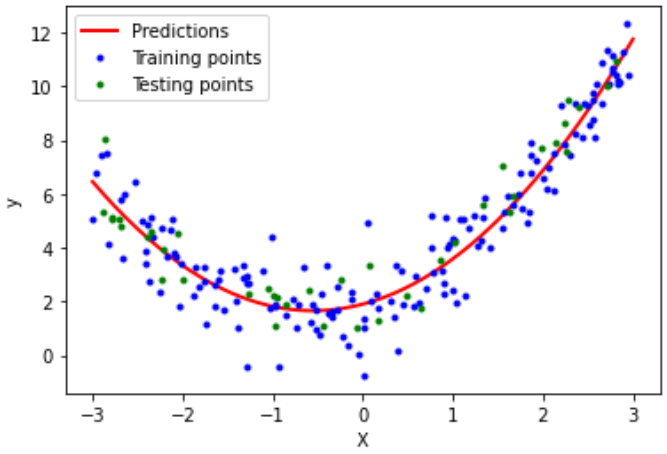
4. Vectorized Gradient:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

5. Gradient Descent Update:

$$\mathbf{w} := \mathbf{w} - \eta \nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{w} - \frac{\eta}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

Polynomial Regression



Polynomial regression is an extension of linear regression where we allow the model to learn curved relationships by adding polynomial terms of the input features.

Model

Instead of fitting a straight line, we fit a polynomial curve. For a single feature x, a degree-k polynomial model is:

$$\mathbf{f}_{\text{aug}} = \begin{bmatrix} 1 \\ f_1 \\ f_2 \\ \dots \\ f_k \end{bmatrix} = \begin{bmatrix} 1 \\ x \\ x^2 \\ \dots \\ x^k \end{bmatrix}$$

$$\hat{y} = \mathbf{w}^T \mathbf{f}_{\text{aug}} = w_0 + w_1 x + w_2 x^2 + \cdots + w_k x^k$$

This allows the model to capture bends and curves in the data.

Matrix for Polynomial Features

For n samples and a polynomial of degree k, the matrix X becomes:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ 1 & x_2 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{bmatrix}$$

Prediction:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

Overfitting: when a model learns the training data too closely and it performs very well on the training data but fails to work well on new or unseen data.

Ridge Regression (L2 Regularization)

Ridge Regression is a regression technique that adds an L2 penalty (sum of squares of coefficients) to the cost function. This modification helps control overfitting and improves the stability of the model.

Problem of Overfitting in Linear and Polynomial Regression

In linear regression, if we have many features or the features are highly correlated, the model can try to fit the training data too closely. This overfitting often results in very large coefficients $w1, w2, \ldots, w_d$, making the model unstable and sensitive to small changes in the input. In polynomial regression, the problem is even more pronounced. Higher-degree polynomials (large k) can fit the training data almost perfectly. While this may reduce training error, it often fails on new data.

Ridge Cost Function

Ridge adds a penalty term proportional to the **square of the coefficients**:

$$J(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \frac{\lambda}{2}\sum_{j=1}^{d} w_j^2$$

- First term → ordinary Mean Squared Error (how far predictions are from actual values)
- Second term → L2 penalty that discourages large coefficients
- λ → regularization strength (controls how much coefficients are shrunk)
- The intercept w_0 is usually not regularized.

Gradient (Vectorized Form)

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}_{\text{reg}}$$

Lesso Regression (L1 Regularization)

Lasso Regression is a type of linear regression that adds an L1 penalty to the cost function. This penalty helps prevent overfitting and can also select important features by setting some coefficients to zero.

Lasso Cost Function

Lasso adds an L1 penalty (sum of absolute values of coefficients):

$$J(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \sum_{j=1}^{d} |w_j|$$

Optimization

- L1 penalty is not differentiable at zero, so Lasso uses:
  - Coordinate Descent or
  - Proximal Gradient Descent (Soft-Thresholding)

Soft-thresholding update for each w_j:

$$w_j = \text{sign}(z_j)\ \max\left(|z_j| - \lambda, 0\right)$$

z_j = partial residual term (tells us how much a feature can improve the model at the current step).

Ridge vs Lasso:

- Ridge → shrinks all coefficients, none are zero
- Lasso → shrinks and can eliminate unimportant features
- Lasso is especially useful when we suspect that only some features are important.